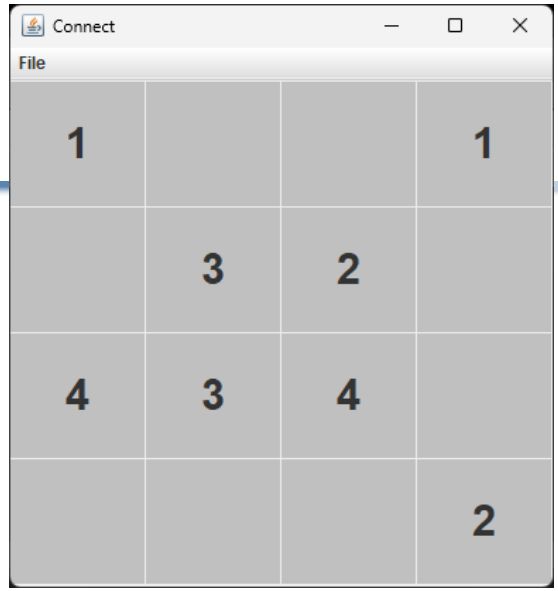
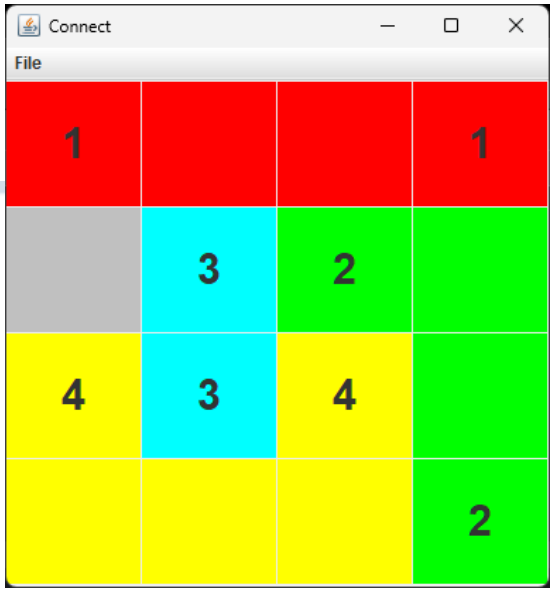
Aufgaben zur Abgabe als Prüfungsleistung für Prog II

- Regeln wie bisher
  - Sprache: Alles Englisch
  - Die Klassen sollen im eigenen Abgabeprojekt in drei Pakete `model`, `view` und `controller` korrekt eingeordnet werden
- Punkteverteilung
  - 12 Punkte Funktionalität (inkl. Anforderungen wie MVC)
  - 8 Punkte Qualität

Connect



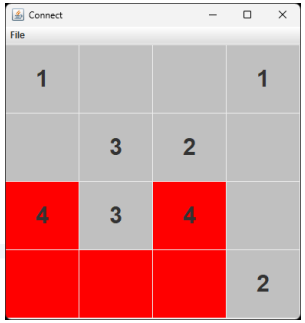


- Spielidee
  - Verbinde zwei passende Nummern mit einem Weg (= Connection, Verbindung)
  - Die Verbindungen dürfen sich nicht kreuzen und nur horizontal bzw. vertikal laufen, aber nicht diagonal
- Bedienung
  - Maustaste auf einer Zahl ist der Start der Verbindung, dann horizontal bzw. vertikal die Zellen durchlaufen, dann auf der Zielnummer die Maustaste loslassen
  - Ein Mausklick auf irgendeiner Zelle einer bestehenden Verbindung löscht die Verbindung
  - Wenn alle Zahlen korrekt verbunden sind, wird automatisch eine Erfolgsmeldung angezeigt („You won!“)

Vorlage

- Es gibt eine lauffähige Vorlage TemplateAssignment2.java im Import-Projekt, diese Klasse bitte als Grundlage nutzen
  - Dort ist z.B. die Nummerierung der Zellen sichtbar und das Mouse-Handling mit den passenden Ereignissen vorhanden.

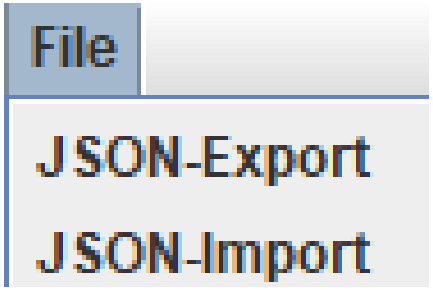
Regeln für Verbindungen



- Mittels `mousePressed` wird eine neue Connection gestartet und alle Punkte von `mouseEntered` zu der aktuellen Connection hinzugefügt, bis `mouseReleased` (= Abschluss der Connection)
- Es dürfen aber keine Punkte als Startpunkt einer Connection aufgenommen werden, die schon in einer anderen Connection enthalten sind; versucht der User dies, werden auch alle folgenden Punkte bis zum Ereignis `mouseReleased` nicht hinzugefügt
- Es dürfen keine Punkte zu einer Connection hinzugefügt werden, die schon in einer anderen Connection enthalten sind
- Löschen von Connections: Bei `mousePressed` und `mouseReleased` auf einem einzigen Punkt, der bereits in einer Connection verwendet wird, wird die gesamte Connection entfernt
- Eine gültige Verbindung enthält genau (!) zwei Punkte aus dem Spielfeld mit identischer Nummer (z.B. die Verbindung von Nummer 1 mit einer anderen Zelle mit derselben Nummer 1). Falls diese Bedingung verletzt ist, wird die Verbindung nicht im Model behalten. Mit anderen Worten: Der erste Punkt der Verbindung muss dieselbe Nummer haben wie der letzte Punkt der Verbindung, dazwischen sind nur Zellen ohne Nummern zulässig.
- Die Connection darf nur horizontal oder vertikal fortgesetzte Punkte enthalten, keine diagonalen Fortsetzungen. Beispiel: 0/2, 0/3, 1/3, 2/3, 2/2 ist korrekt, aber nicht 0/2, 1/3, 2/3, 2/2
- Hinweis: Verwenden Sie als Datentyp für die Koordinaten auf dem Spielfeld die Klasse `java.awt.Point`. Der Zugriff geht direkt mittels x- und y-Koordinate, z.B. `point.x` bzw. `point.y`

Export/Import

- Menü
  - Es soll in einem File-Menü zwei Funktionalitäten geben: Einen JSON-Export (mittels Save-FileChooser) und einen JSON-Import (mittels Open-FileChooser)
  - Durch den JSON-Import wird nach einem JFileChooser-Dialog eine JSON-Datei (siehe Vorlage, z.B. 10x10.json) importiert und spielfertig vorbereitet
  - Ein File-Chooser-Dialog ist also sowohl beim Export als auch beim Import aufzunehmen.



Vorgehen und weitere Anforderungen an die Implementierung

- Vorgehen
  - Verwenden Sie die Vorlage `mvc_vorlage.zip` aus OLAT für das MVC-Muster und verteilen die vier Dateien auf die drei Pakete `model, view, controller`
  - Verwenden Sie die Vorlage `TemplateAssignment2.java` aus der Aufgabenstellung und verteilen den Inhalt auf die passenden Klassen in diesen Paketen
    - Beispiel: die Konstante `MAX` als änderbares Attribut `max` in die Klasse `Model.java`, die Erzeugung der Oberfläche mit GridLayout usw. in die View, das Handling der Maus von der inneren Klasse `ControllerMouse` in eine separate Controller-Klasse `ControllerMouse` an Stelle der Klasse `ControllerButton`
  - Die Auslieferung soll eine voreingestellte Spielfeldgröße `max = 4` starten; deshalb erzeugen Sie beim Start der Anwendung hardcodiert ein Spielfeld, das die Werte aus Folie 2 enthält

Vorgehen und weitere Anforderungen an die Implementierung

- Vorgehen
  - Erstellen Sie weitere Model-Klassen, u.a. eine Klasse `Connection.java` zum Speichern einer Liste von Punkten, die vom Start-Klick bis zum Loslassen der Maus durchlaufen werden. Objekte dieser Klasse `Connection` werden in einer Liste von Connection-Objekten in der Model-Klasse verwaltet.
  - Sorgen Sie dafür, dass die Ereignisse aus `ControllerMouse` im Model gespeichert werden.
  - Erweitern Sie die View-Klasse, so dass die im Model verwalteten Connections mit farbig markierten Kästchen aus dem Model gelesen und in der View angezeigt werden
  - Es sollen 8 Farben in einem enum *`GameColor.java`* definiert werden
    - Jede neue Connection erhält automatisch die nächste neue Farbe
    - Das enum kann um weitere Farben ergänzt werden ohne dass Code-Anpassungen in anderen Klassen notwendig sind
    - Wenn die Anzahl der Farben ausgeschöpft ist, beginnen die Farben wieder von vorne
    - Hinweis: https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html Beispiel Planet
    - Hinweis: Zugriff mittels Index möglich: *`GameColor.values()[index]`*

Vorgehen und weitere Anforderungen an die Implementierung: Export/Import

- Es gibt ein Beispiel-Spielfeld `10x10.json`, das vom User als neues Spiel importiert werden kann
- Für JSON mittels Jackson muss aus dem Model das Spielfeld in eine separate Klasse `GameBoard.java` ausgelagert werden (und als Attribut, d.h. Komposition, wieder in das Model aufgenommen werden). Der Grund ist, dass dann der JSON-Export/Import genau die gewünschten Attribute der Klasse `GameBoard` verwendet: Die Größe des Spielfelds `max` und das zweidimensionale Feld `fld` von Integer-Zahlen.
- Import und Export sollen mittels FileChooser-Dialog ablaufen.
- Der FileChooser soll auf `.json`-Dateien filtern und beim Speichern die Endung `.json` ergänzen, falls der Nutzer dies nicht eingetippt hat.
- Als Startverzeichnis soll der FileChooser das Home-Verzeichnis verwenden: `FileSystemView.getFileSystemView().getHomeDirectory()`
- Hinweis: Beim Erzeugen eines neu geladenen Spiels muss kein komplett neues Model mittels `new` erzeugt werden. Stattdessen genügt es, nur die Inhalte des Models zu ändern (z.B. leere Connection-Liste, neues GameBoard). Auch in der View muss kein komplett neues View-Objekt erzeugt werden, stattdessen nur die Inhalte aus dem aktualisierten Model mittels einer im Template bereits vorhandenen Methode `createViewForGameBoard()` neu zeichnen.
- Für JSON sind die drei Jackson-jar-Dateien im Import-Projekt als Vorlage enthalten und in den Build-Path des Projekts vorhanden